Benchmarks on Toubkal GPUs

Mohamed Karim Bezine

June 17, 2025

In this report, we will present the steps and the results of running benchmarks on the Toubkal Supercomputer GPUs.
Important Note: All benchmarks presented in this report were executed on a single GPU.

1 A100 GPUs

1.1 NVIDIA NGC Container for Benchmarking on GPUs

NVIDIA's NGC (NVIDIA GPU Cloud) provides a catalog of GPU-optimized containers designed for AI, HPC and scientific computing. These containers are pre-configured with the necessary libraries and dependencies ensuring a stable and optimized environment for benchmarking without the hassle of manual setup.
Therefore, we used the NVIDIA HPC Benchmarking container from NGC, which includes optimized implementations of ***HPL*** and ***STREAM***. Using NGC allowed us to focus on performance evaluation and system tuning while ensuring consistency across different GPU-based platforms.

1.2 Execution steps

In this section, we will outline the essential execution steps required for running all benchmarks. These steps ensure proper setup and execution of the NVIDIA HPC Benchmarks. The benchmark images are available from the NVIDIA NGC catalog. To pull the NVIDIA HPC Benchmark image using Singularity, execute the following command. In our case, we pull the 23.10 version:

```
$ singularity pull docker://nvcr.io/nvidia/hpc-benchmarks:23.10
```

Now, we need to navigate to our working directory and set up the necessary files for running the benchmarks. The first step is to create a dedicated directory named `data`, which will store configuration files required for execution. We will then create an `HPL.dat` file inside this directory. Follow the steps below:

1. Navigate to your desired working directory (replace `/path/to/working/directory` with the actual location):

   ```
   $ cd /path/to/working/directory
   ```

2. Create a new directory named `data` to store benchmark-related files:

   ```
   $ mkdir data
   ```

3. Move into the newly created `data` directory:

   ```
   $ cd data
   ```

4. Create an empty `HPL.dat` file, which will store the configuration parameters for the benchmark:

```
$ touch HPL.dat
```

5. Open the `HPL.dat` file using a text editor (e.g., Vim) to modify and define the necessary parameters:

```
$ vim HPL.dat
```

6. Inside `HPL.dat`, you will need to configure parameters such as problem size, process grid dimensions, and block size. These settings are crucial for optimizing performance based on hardware specifications.

After setting up the `HPL.dat` file, we will proceed with executing the benchmark using the appropriate configurations. The `HPL.dat` file contains key parameters that define the execution settings of the benchmark, including problem sizes, block sizes, process grid dimensions, and algorithmic parameters. These configurations must be carefully selected based on the available hardware resources to optimize performance.

Below is the content for the `HPL.dat` file:

```
HPLinpack benchmark input file
Innovative Computing Laboratory, University of Tennessee and Frankfurt
    Institute for Advanced Studies
HPL.out      output file name (if any)
6            device out (6=stdout,7=stderr,file)
5         # of problems sizes (N)
20000 40000 60000 80000 100000        Ns
1            # of NBs
576          NBs
1            PMAP process mapping (0=Row-,1=Column-major)
1            # of process grids (P x Q)
1            Ps
1            Qs
0.1          threshold
1            # of panel fact
1            PFACTs (0=left, 1=Crout, 2=Right)
1            # of recursive stopping criterium
64           NBMINs (>= 1)
1            # of panels in recursion
2            NDIVs
1            # of recursive panel fact.
0            RFACTs (0=left, 1=Crout, 2=Right)
1            # of broadcast
6            BCASTs (0=1rg,1=1rM,2=2rg,3=2rM,4=Lng,5=LnM,6=MPI)
1            # of lookahead options
3            LOOKAHEADs (enable = 1)
8            memory alignment in double (> 0)
100          Seed for the matrix generation
```

1.2.1 Explanation of Key Parameters

- **Problem Sizes (N):** Specifies the matrix sizes for benchmarking. The five values provided (`20000, 40000, 60000, 80000, 100000`) allow testing system performance at different scales.

- **Block Size (NBs):** Defines the computational block size. Here, `576` is chosen to balance workload distribution and optimize GPU performance.

- **Process Grid (P × Q):** Determines the number of processes in the grid.
  - `Ps = 1`, `Qs = 1` → A single process grid is used.
  - The total number of processes is `P × Q`.
- **Threshold Value:** `0.1` is set as the threshold value, which serves as a stopping criterion for residual checks.

1.2.2 Binding Directories to Singularity

In this section, we bind a local directory on the host machine to a directory inside the Singularity container. This ensures that files like 'HPL.dat', which reside on the host system, are accessible within the container.

```
$ pwd
/home/karim.bezine-ext/lustre/sw_stack-373lcd9r8io/users/karim.bezine-ext/src/
    data
```

The 'pwd' command prints the current working directory. Here, it is assumed that the directory containing the necessary data file ('HPL.dat').
Now, we bind this directory to the container:

```
$ singularity exec --bind /home/karim.bezine-ext/lustre/sw_stack-373lcd9r8io/
    users/karim.bezine-ext/src/data:/mnt hpc-benchmarks_23.10.sif ls /mnt/HPL.
    dat
```

The 'singularity exec' command runs a command inside the Singularity container. The '–bind' flag binds the local directory to the container's '/mnt' directory. Afterward, 'ls /mnt/HPL.dat' lists the 'HPL.dat' file inside the container to confirm the binding.
If the session is closed, we will need to re-enter the bind command every time. To avoid this, we can make the bind persistent by adding the bind command to the '.bashrc' file, which is executed every time a new terminal session is opened.

```
$ vim ~/.bashrc
```

Add the following line to bind the directory persistently:

```
$ export SINGULARITY_BINDPATH=/home/karim.bezine-ext/lustre/sw_stack-373
    lcd9r8io/users/karim.bezine-ext/src/data:/mnt
```

This command adds an environment variable binding the host directory to the container each time a terminal session starts.

```
$ source ~/.bashrc
```

1.2.3 Allocating GPU Resources

Before running any GPU-accelerated benchmarks, you need to allocate GPU resources. This step ensures that the system allocates a node with a GPU available for your benchmark. The NVIDIA driver should be displayed after running those commands.

```
$ salloc -t 4:00:00 -n 1 -A sw_stack-373lcd9r8io-default-gpu -p gpu --gres=gpu
    :1
$ singularity run--nv hpc-benchmarks_23.10.sif
$ nvidia-smi
```

1.3 HPL High Performance Linpack

High-Performance Linpack (HPL) is a widely used benchmark for measuring the floating-point performance of supercomputers, particularly in the TOP500 rankings. Traditionally optimized for CPUs, HPL has been adapted for GPUs to leverage their massive parallel processing capabilities. GPU-accelerated versions utilize optimized libraries like cuBLAS to achieve high computational efficiency, making them essential for evaluating modern HPC systems with NVIDIA GPUs.

1.3.1 Running the HPL Benchmark

To run the HPL (High Performance Linpack) benchmark, follow the steps below:

1. Navigate to the directory where the HPL benchmark files are located:

```
$ cd ../workspace/hpl-linux-x86_64/
```

2. Run the benchmark using MPI. The command below specifies the number of processes (‘-np 1‘ for a single process) and the location of the configuration file (‘HPL.dat‘) that will be used for the benchmark execution:

```
$ mpirun -np 1 hpl.sh --dat /mnt/HPL.dat --gpu-affinity 0 --no-multinode --cuda
    -compat
```

- **mpirun -np 1**: Runs an MPI process with a single instance.
- **hpl.sh**: The script that launches the HPL-NVIDIA benchmark.
- **–dat /mnt/HPL.dat**: Specifies the path to the HPL.dat configuration file.
- **–gpu-affinity 0**: Ensures execution runs on GPU 0.
- **–no-multinode**: Disables multi-node execution for a single-node setup.
- **–cuda-compat**: Enables CUDA forward compatibility for running binaries compiled with older CUDA versions.

3. Once the command is executed, the benchmark will begin its execution, and you'll see output that provides details about the progress of the benchmark.

The benchmark runs the HPL test, performing matrix multiplications to measure the system's floating-point performance. The ‘HPL.dat‘ file contains configuration settings, such as problem size and process grid dimensions, which affect the benchmark's performance.

1.3.2 HPL Performance Results Evaluation

Below is a table summarizing the results from five different test cases with varying problem sizes (‘N‘). The results include the time taken for each test and the achieved computation speed in Gflops (Billions of Floating-Point Operations Per Second):

This table demonstrates how the time taken and the computation speed (in Gflops) change as the problem size (‘N‘) increases. It is important to note that larger problem sizes (higher ‘N‘ values) typically result in higher computation speeds, but they also take longer to process.
The NVIDIA NGC HPL benchmark is optimized to use FP64 Tensor Cores by default when running on A100-SXM4-80GB GPUs. These specialized cores significantly boost double-precision floating-point performance, enabling higher efficiency in large-scale numerical computations.
For an A100 GPU, the expected performance is around 19.2 TFLOPS per GPU. Our benchmark results, reaching approximately 92% of this theoretical peak (∼18 TFLOPS), confirm that the system is performing as expected.

Problem Size, N	Time Taken (s)	Computation Speed (Gflops)
20000	0.51	9731
40000	2.63	15890
60000	8.36	17150
80000	19.02	17600
100000	36.94	17860

Figure 1: HPL Results

1.3.3 Graphical Representation of Results

The following figure illustrates the relationship between problem size ('N') and computation speed in Gflops. As seen in the graph, the computation speed tends to increase as the problem size grows, which is expected for large-scale numerical problems handled by GPUs.

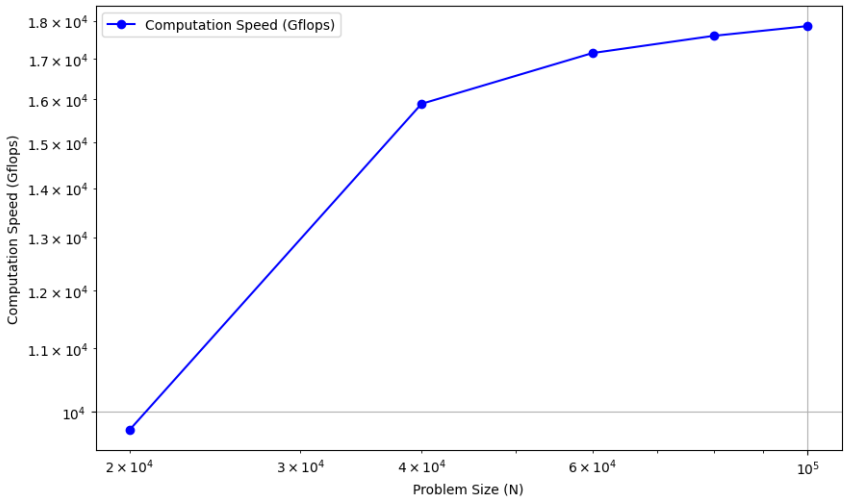


Figure 2: Computation Speed vs. Problem Size

The graph above shows the trend of computation speed as a function of the problem size for a single GPU on the Toubkal cluster. As you can observe, the computation speed increases as the problem size ('N') grows. This indicates that the GPU is effectively utilizing its computational power for larger problem sizes.
Important Note: This occurs because the system does not have sufficient memory to handle such a large matrix size, and the benchmark will fail with an error message indicating insufficient resources. To avoid this, ensure that the problem size is within the supported memory limits of your GPU.

1.4 STREAM Benchmark

The **STREAM benchmark** is a synthetic memory bandwidth test that measures sustained memory performance through four vector operations (Copy, Scale, Add, and Triad), with Triad being the most relevant for bandwidth evaluation. The problem size (N) significantly impacts results, as it must be large enough to avoid cache effects and fully utilize GPU memory bandwidth.

1.4.1 Understanding the value of n

- These arrays are used for the Copy, Scale, Add, and Triad operations, and each element in these arrays represents a data point on which the operation will be applied.

- For double-precision (FP64) operations, each element consumes 8 bytes of memory.
- The size of each array is determined by $n \times 8 \times 8$ bytes, where:
  - The first 8 accounts for the 8 bytes required to store a single double-precision element.
  - The second 8 indicates that each array element is stored as a vector (or block) of 8 double-precision numbers, so each vector occupies $8 \times 8 = 64$ bytes.
- In our case, n is set to 1,300,000,000, it allocates an array size of approximately 79,345 MB (roughly 80 GB), which fits within the available memory of the NVIDIA A100 80GB GPU.
  Important Note: If the problem size n exceeds $13 * 10^8$, you will encounter an out-of-memory error.

1.4.2 Navigate to the STREAM Benchmark Directory

```
$ cd ../stream-gpu-linux-x86_64/
```

1.4.3 Running the Benchmark

After setting an appropriate n value based on available GPU memory, run the benchmark with:

```
$ mpirun -np 1 ./stream-gpu-test.sh --n 1300000000
```

STREAM Performance Results Evaluation Below is a table summarizing the STREAM benchmark results for the NVIDIA A100-SXM4-80GB:

Function	Rate (MB/s)	Avg Time (s)	Min Time (s)	Max Time (s)
Copy	1783519.13	0.0117	0.0117	0.0117
Scale	1790751.93	0.0116	0.0116	0.0116
Add	1805341.67	0.0174	0.0173	0.0174
Triad	1806733.29	0.0174	0.0173	0.0174

Figure 3: NVIDIA STREAM Benchmark Results for Various Operations

Those resuslts indicate that the Triad operation achieved approximately 1.8 TB/s of memory bandwidth. Given that the theoretical peak bandwidth for this GPU is 2 TB/s, our measured performance is quite close to the expected value.

2 H100 GPUs

2.1 Overview

The H100 GPU, based on NVIDIA's Hopper architecture, provides significant advancements over the A100 GPU, offering more CUDA cores, higher memory bandwidth, and architectural improvements specifically aimed at accelerating both AI and HPC workloads. Benchmarking this new architecture provides insight into performance gains and potential tuning strategies compared to A100 GPUs.

Similar to our setup for the A100, we utilized the NVIDIA HPC Benchmarking container from NGC to evaluate the system performance. The same benchmark tools **HPL** and **STREAM** were executed on a single H100 GPU.

2.2 Execution Environment

We used the same containerized setup via Singularity to ensure reproducibility and consistency across benchmark runs. The following Singularity command was used to execute the container with H100 GPU access:

```
$ salloc -A SW_STACK-373LCD9R8IO-DEFAULT-GPU -p gpu_h100 --time 01-00:00:00 --
    qos premium-gpu-temp --gres gpu:1
```

The entire benchmarking procedure for the H100 was kept identical to that used for the A100, with the only modification being the SLURM command—specifically, the partition name changed from `gpu_a100` to `gpu_h100` to reflect the targeted hardware. All other steps, including the use of the NVIDIA HPC Benchmarking container, invocation of **HPL** and **STREAM** benchmarks, and post-processing, remained the same. This ensures a fair comparison and allows for a seamless re-execution of the A100 steps with H100 hardware.

However, while the procedure remains unchanged, the results are not the same. Due to the architectural enhancements and hardware improvements of the H100 GPU, performance metrics such as floating-point operations per second and memory throughput show significant differences when compared to the A100.

2.3 HPL High Performance Linpack

2.3.1 HPL Performance Results Evaluation

To evaluate the performance differences between NVIDIA A100 and H100 GPUs, the High-Performance Linpack (HPL) benchmark was executed using various problem sizes ranging from 20,000 to 100,000. Two primary metrics were analyzed: **execution time** and **computation speed (in Gflops)**.

2.3.2 Execution Time (H100 Only)

Problem Size, N	Time Taken (s)	Computation Speed (Gflops)
20000	0.31	16,230
40000	1.17	35,760
60000	3.43	41,760
80000	7.57	44,230
100000	12.43	45,210

Figure 4: HPL Results on H100

2.3.3 Computation Speed Comparison Between A100 and H100

Problem Size, N	A100 (Gflops)	H100 (Gflops)
20000	9,731	16,230
40000	15,890	35,760
60000	17,150	41,760
80000	17,600	44,230
100000	17,860	45,210

Figure 5: Computation Speed Comparison Between A100 and H100

2.4 Discussion

The results from the HPL benchmark clearly demonstrate the performance superiority of the NVIDIA H100 GPU compared to the A100 across all tested problem sizes.

2.4.1 Performance Interpretation

- **Higher Gflops Throughput:** The H100 consistently delivers significantly higher Gflops than the A100, starting from a 1.7× improvement at $N = 20000$ to a 2.53× improvement at $N = 100000$. This indicates that the H100 executes floating-point operations more efficiently and with higher throughput.
- **Better Scalability:** The H100 shows better scalability with increasing problem size. While the A100's performance saturates around 17,800 Gflops, the H100 continues to scale linearly, achieving over 45,000 Gflops. This suggests that the H100 handles large-scale linear algebra computations with greater efficiency, making it ideal for HPC workloads.
- **Improved Architecture:** The performance improvements can be largely attributed to architectural advancements in the Hopper-based H100 over the Ampere-based A100:
  - **Tensor Core Enhancements:** The H100 includes fourth-generation Tensor Cores that support a broader range of data types (including FP8) and offer higher throughput per SM compared to the A100's third-generation Tensor Cores.
  - **Increased SM Count and Clock Speeds:** The H100 includes more Streaming Multiprocessors (SMs) and operates at higher clock frequencies, directly increasing parallelism and compute density.
  - **Transformer Engine:** While not directly exploited in HPL, the H100's Transformer Engine accelerates matrix-heavy operations, which contributes to more efficient execution of large matrix solves like those in HPL.
  - **Memory Bandwidth and Latency Improvements:** The H100 offers increased memory bandwidth, reducing bottlenecks for large matrix data movement, which is critical in memory-bound operations like those in HPL.
- **Reduced Execution Time:** As shown in Table 4, execution times on the H100 remain relatively low even for large problem sizes. For instance, solving a problem of size $N = 100000$ takes only 12.43 seconds on the H100. This is particularly relevant in HPC environments where reducing time-to-solution is crucial.
- **Energy Efficiency:** Although not explicitly measured in this benchmark, the H100 is expected to provide higher performance-per-watt compared to the A100. This makes it more suitable for large-scale deployments where power efficiency is a significant factor.
- **Future-Proofing and Software Stack Optimization:** The H100 benefits from the latest NVIDIA software stack improvements, such as CUDA 12+, optimized cuBLAS libraries, and Hopper-specific tuning in frameworks like HPC-Bench. These enhancements further boost performance when compared to legacy support for the A100.

2.4.2 Graphical Representation of Results

The graphical plot in Figure 6 visually reinforces the numerical trends. The H100 curve rises steeply and consistently, showing superior linear scalability. In contrast, the A100's performance flattens beyond $N = 60000$, indicating limitations in either memory bandwidth or architectural saturation.

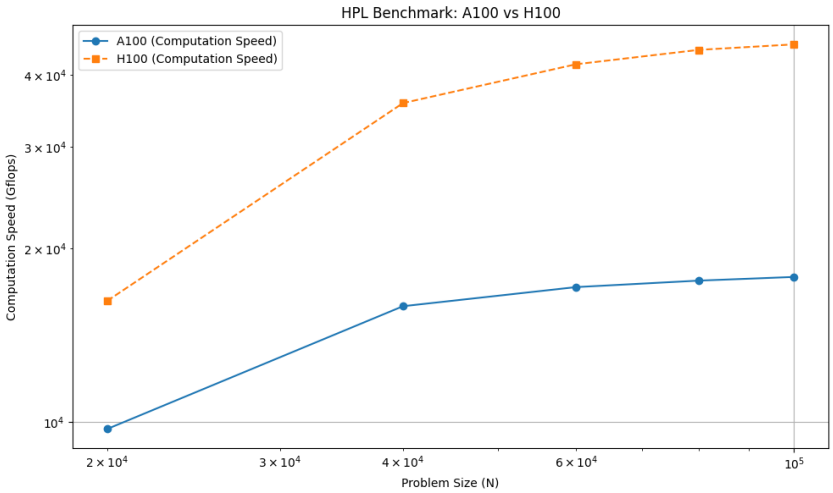


Figure 6: Computation Speed vs. Problem Size for A100 and H100

Overall, these results validate the H100 GPU as a more powerful and scalable solution for high-performance scientific computing tasks, especially in domains relying heavily on dense linear algebra such as climate modeling, computational chemistry and AI training at scale.

2.5 STREAM Benchmark

2.5.1 Memory Bandwidth Evaluation on H100

To assess the memory subsystem performance of the NVIDIA H100 GPU, we conducted the STREAM benchmark, which evaluates sustainable memory bandwidth using four key vector operations: `Copy`, `Scale`, `Add`, and `Triad`. These operations stress the memory bandwidth rather than the compute units, making STREAM an effective benchmark for memory-bound HPC applications.

2.5.2 STREAM Benchmark Results on H100

Function	Rate (MB/s)	Avg Time (s)	Min Time (s)	Max Time (s)
Copy	3,056,220.41	0.0068	0.0068	0.0068
Scale	3,073,925.53	0.0068	0.0068	0.0068
Add	3,132,077.26	0.0100	0.0100	0.0110
Triad	3,131,473.77	0.0100	0.0100	0.0111

Figure 7: STREAM Benchmark Results on NVIDIA H100

2.5.3 Discussion

The STREAM benchmark results for the H100 GPU indicate exceptional memory performance, achieving bandwidths in excess of 3,100 GB/s for vector operations like `Add` and `Triad`. Key takeaways include:

- **Consistent Memory Throughput:** All four operations consistently deliver over 3,000 GB/s, with minimal variance across average, minimum, and maximum times. This consistency indicates highly stable memory performance on the H100.

- **Peak Bandwidth Utilization:** The `Add` and `Triad` operations, which involve three vector operands, achieve the highest bandwidth rates, peaking at approximately 3,132 GB/s. This shows the H100's ability to effectively handle multi-stream memory transactions.
- **Latency Stability:** The near-identical average, min, and max timings for `Copy` and `Scale` (both 0.0068s) suggest minimal jitter and highly optimized memory access paths.
- **Architectural Impact:** The high performance in STREAM is attributable to improvements in the H100's Hopper architecture, particularly:
  - **Improved Memory Subsystem:** Enhanced HBM3 memory and a wider memory interface boost effective bandwidth.
  - **Higher Memory Clock Speeds:** These contribute to faster data movement across the memory hierarchy.
  - **Optimized Cache Hierarchy:** Improved L2 cache utilization and higher cache bandwidth reduce pressure on global memory access.
- **Implications for HPC:** Applications that are memory-bound — such as stencil codes, sparse matrix computations, and finite-difference solvers — are expected to benefit significantly from this level of bandwidth performance. It positions the H100 as a top-tier candidate for data-intensive HPC workloads.

In summary, the STREAM benchmark results further reinforce the H100's suitability for high-performance computing environments, showcasing its ability to deliver both compute power (as seen in HPL) and memory throughput at scale.

References

1. **Sylabs.** *Binding Directories and Mounts*. Available at: Singularity Official Documentation.

2. **HPL.** *High Performance Linpack*. HPL Results.

3. **STREAM Benchmark.** *Memory Bandwidth Benchmark*. STREAM Results.